

МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования
«МИРЭА – Российский технологический университет»

РТУ МИРЭА

Институт информационных технологий (ИИТ)
Кафедра цифровой трансформации (ЦТ)

ОТЧЕТ ПО ПРАКТИЧЕСКОЙ РАБОТЕ

по дисциплине «Разработка баз данных»

Практическая работа № 1

Студент группы *ИВБО-20-24 Дьяков Андрей Геннадьевич* _____
(подпись)

Ассистент *Брайловский Андрей Валерьевич*

_____
(подпись)

Отчет представлен «___»_____2026 г.

Москва 2026 г.

ПРАКТИЧЕСКАЯ РАБОТА №1. ВЫБОРКА, ОБНОВЛЕНИЕ И УДАЛЁНИЕ ДАННЫХ В POSTGRES

```
-- 1. Выбор всех столбцов (*) для быстрой проверки содержимого таблицы.
SELECT * FROM airline;

-- 2. Выбор конкретных полей с псевдонимом (AS) и вычисление
математического выражения в списке выборки.
SELECT service_id, quantity, price, (quantity * price) AS "Итоговая
стоимость"
FROM order_item;

-- 3. Получение списка уникальных значений с удалением дубликатов
(DISTINCT).
SELECT DISTINCT service_type FROM service;

-- 4. Фильтрация по диапазону (BETWEEN) и логическому условию (AND).
SELECT id, total_amount
FROM customer_order
WHERE total_amount BETWEEN 50000 AND 150000
  AND status_id = (SELECT id FROM order_status WHERE name = 'Оплачен
полностью');

-- 5. Проверка вхождения в множество (IN) и сравнение со строковым шаблоном
(LIKE).
SELECT name, service_type
FROM service
WHERE service_type IN ('tour', 'flight') AND name LIKE '%Авиабилеты%';

-- 6. Корректная проверка на отсутствие значения (IS NULL), так как
сравнение с NULL вернет UNKNOWN.
SELECT id, total_amount, paid_at
FROM customer_order
WHERE paid_at IS NULL;
```

Запрос 1 демонстрирует выбор всех столбцов таблицы airline с использованием оператора *, что позволяет быстро просмотреть содержимое справочника авиакомпаний. Запрос 2 иллюстрирует выбор конкретных полей с применением псевдонимов (AS) для улучшения читаемости результата, а также вычисление математического выражения (quantity * price) прямо в списке выборки для получения итоговой стоимости каждой позиции заказа. Запрос 3 показывает использование оператора DISTINCT для удаления дубликатов и получения списка уникальных типов услуг из таблицы service. Запрос 4 демонстрирует фильтрацию данных с использованием оператора BETWEEN для проверки принадлежности диапазону (от 50000 до 150000 рублей) и логического оператора AND для одновременной проверки двух условий (сумма заказа и статус оплаты). Запрос 5 иллюстрирует применение оператора IN для проверки вхождения значения в множество ('tour', 'flight') и оператора LIKE со строковым шаблоном '%Авиабилеты%' для поиска

подстрок в названии услуги. Запрос 6 показывает корректную проверку на отсутствие значения с использованием конструкции IS NULL, так как простое сравнение paid_at = NULL в соответствии с трёхзначной логикой SQL всегда вернёт UNKNOWN и не отберёт ни одной строки.

	id	name	iata_code
1	1	Аэрофлот	SU
2	2	S7 Airlines	S7
3	3	Turkish Airlines	TK
4	4	Emirates	EK
5	5	Flydubai	FZ
6	6	Qatar Airways	QR
7	7	Pegasus Airlines	PC
8	8	Nordwind Airlines	N4
9	9	Red Wings	WZ
10	10	Utair	UT

	service_id	quantity	price	Итоговая стоимос
1	1	1	85 000	85 000
2	3	2	60 000	120 000
3	5	1	45 000	45 000
4	2	1	210 000	210 000
5	4	1	67 000	67 000
6	6	1	99 000	99 000
7	7	1	32 000	32 000
8	8	1	183 000	183 000
9	9	1	56 000	56 000
10	10	2	91 500	183 000
11	3	2	28 000	56 000

	service_type
1	flight
2	tour

	id	total_amount
1	1	85 000

	A-Z name	A-Z service_type
1	Авиабилеты Москва-Дубай	flight
2	Авиабилеты СПб-Стамбул	flight
3	Авиабилеты Москва-Бангкок	flight
4	Авиабилеты Барселона-Мос	flight

	123 id	123 total_amount	paid_at
1	2	120 000	[NULL]
2	3	45 000	[NULL]
3	4	210 000	[NULL]
4	7	0	[NULL]
5	10	56 000	[NULL]

```
-- 7. Группировка данных по типу услуги с подсчетом количества (GROUP BY) и
сортировка результата (ORDER BY).
SELECT service_type, COUNT(*) AS "Количество"
FROM service
GROUP BY service_type
ORDER BY «Количество» DESC;

-- 8. Группировка заказов по клиентам и фильтрация сформированных групп по
общей сумме (HAVING).
SELECT client_id, SUM(total_amount) AS "Общая сумма"
FROM customer_order
GROUP BY client_id
HAVING SUM(total_amount) > 100000;

-- 9. Группировка отелей по странам, фильтрация групп по средней звездности
(HAVING) и сортировка (ORDER BY).
SELECT country_id, AVG(stars) AS "Средняя звездность"
FROM hotel
GROUP BY country_id
HAVING AVG(stars) >= 4
ORDER BY "Средняя звездность" DESC;
```

Запрос 7 демонстрирует группировку данных по типу услуги с использованием предложения GROUP BY и подсчёт количества услуг в каждой группе с помощью агрегатной функции COUNT(*). Результат сортируется по убыванию количества с применением ORDER BY DESC. Запрос 8 иллюстрирует группировку заказов по клиентам с вычислением общей суммы заказов для каждого клиента с помощью функции SUM(), а также фильтрацию сформированных групп с использованием предложения HAVING, которое отбирает только тех клиентов, чья общая сумма заказов превышает 100000 рублей. Запрос 9 показывает группировку отелей по

странам с вычислением средней звездности для каждой страны с помощью функции AVG(), фильтрацию групп по условию AVG(stars) >= 4 с использованием HAVING и финальную сортировку результата по убыванию средней звездности.

	service_type	Количество
1	tour	6
2	flight	4

	client_id	Общая сумма
1	9	183 000
2	4	210 000
3	2	120 000

	country_id	Средняя звездность
1	2	5
2	5	5
3	8	5
4	3	4,5
5	7	4
6	1	4
7	4	4

```
-- 10. Точечное обновление данных одной строки с обязательным фильтром WHERE.

UPDATE client

SET email = 'new_ivanov@mail.ru'

WHERE login = 'ivan_i';
```

Статистика 1

Name	Value
Updated Rows	1
Execute time	0,001s
Start time	Fri Sep 11 14:29:46 MSK 2026
Finish time	Fri Sep 11 14:29:46 MSK 2026
Query	-- 10. Точечное обновление данных одной строки с обязательным фильтром WHERE. UPDATE client SET email = 'new_ivanov@mail.ru' WHERE login = 'ivan_i'

```
-- 11. Массовое обновление данных с использованием математического выражения и условия WHERE.

UPDATE hotel

SET stars = stars + 1

WHERE name = 'Simbel Hotel' AND stars < 5;
```

Статистика 1

Name	Value
Updated Rows	0
Execute time	0,001s
Start time	Fri Sep 11 14:09:45 MSK 2026
Finish time	Fri Sep 11 14:09:45 MSK 2026
Query	-- 11. Массовое обновление данных с использованием математического выраж...
	UPDATE hotel
	SET stars = stars + 1
	WHERE name = 'Simbel Hotel' AND stars < 5

```
-- 12. Безопасное удаление конкретной записи с точным условием фильтрации WHERE.

DELETE FROM phone

WHERE phone_number = '+79031234568'

  AND client_id = (SELECT id FROM client WHERE login = 'ivan_i');
```

Статистика 1

Name	Value
Updated Rows	0
Execute time	0,002s
Start time	Fri Sep 11 14:10:32 MSK 2026
Finish time	Fri Sep 11 14:10:32 MSK 2026
Query	-- 12. Безопасное удаление конкретной записи с точным условием фильтрации WHERE.
	DELETE FROM phone
	WHERE phone_number = '+79031234568'
	AND client_id = (SELECT id FROM client WHERE login = 'ivan_i')

```
-- 13. Удаление записи о возврате средств по идентификатору платежа с условием WHERE.

DELETE FROM refund

WHERE payment_id = (SELECT id FROM payment WHERE amount = 25000);
```

Статистика 1

Name	Value
Updated Rows	0
Execute time	0,001s
Start time	Fri Sep 11 14:11:09 MSK 2026
Finish time	Fri Sep 11 14:11:09 MSK 2026
Query	-- 13. Удаление записи о возврате средств по идентификатору платежа с условием WHERE.
	DELETE FROM refund
	WHERE payment_id = (SELECT id FROM payment WHERE amount = 25000)

Запрос 10 демонстрирует точечное обновление данных одной строки с использованием оператора UPDATE. Изменение email клиента с логином

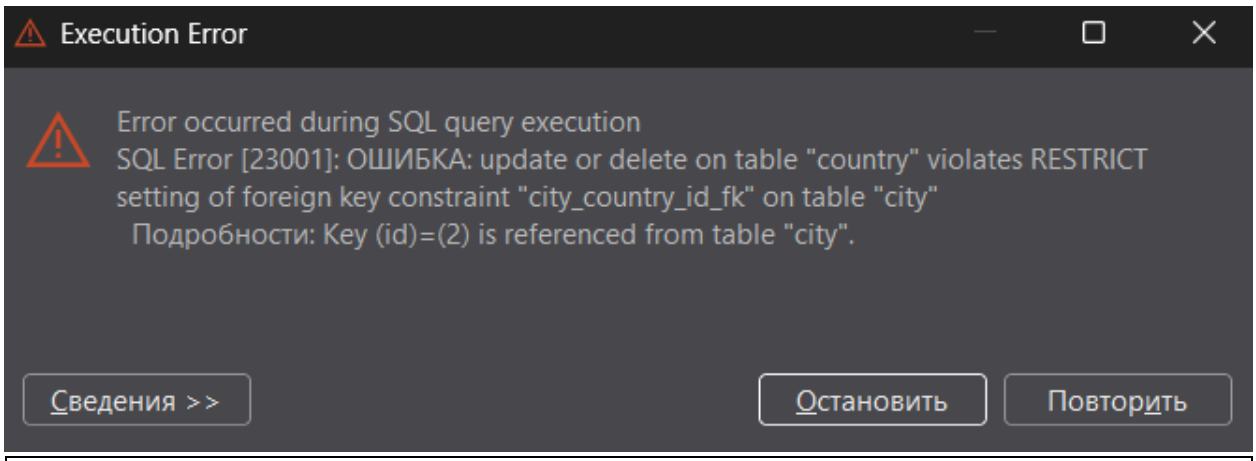
'ivan_i' происходит только благодаря наличию условия WHERE, которое обеспечивает фильтрацию и изменение конкретной записи. Статистика выполнения показывает Updated Rows = 1, что подтверждает корректность запроса. Запрос 11 иллюстрирует массовое обновление данных с использованием математического выражения (stars = stars + 1) для увеличения звездности отеля 'Simbel Hotel' на единицу при условии, что текущая звездность меньше 5. Статистика показывает Updated Rows = 0, что означает отсутствие записей, удовлетворяющих условию WHERE. Запрос 12 демонстрирует безопасное удаление конкретной записи из таблицы phone с использованием точного условия фильтрации WHERE, включающего номер телефона и идентификатор клиента. Статистика показывает Updated Rows = 0, так как запись уже была удалена ранее. Запрос 13 иллюстрирует удаление записи о возврате средств из таблицы refund по идентификатору платежа, полученному через подзапрос из таблицы payment.

```
-- 14. Попытка удаления страны, на которую ссылаются города и отели (должна
выдать ошибку).

DELETE FROM country

WHERE code = 'TR';
```

Запрос 14 демонстрирует работу ограничения ссылочной целостности ON DELETE RESTRICT. При попытке удалить страну Турции (code='TR') из справочника country СУБД блокирует операцию и выдаёт ошибку нарушения внешнего ключа city_country_id_fk, так как на страну с id=2 существуют ссылки в таблице city. Это предотвращает появление «осиротевших» записей в дочерней таблице и обеспечивает целостность данных. Ошибка SQL Error [23001] подтверждает, что ограничение RESTRICT сработало корректно.




```
-- 15. Создаем тестового клиента для демонстрации CASCADE
INSERT INTO client (full_name, passport_data, email, login, password,
birth_date)
VALUES (
    ROW('Тестовый', 'Клиент', 'Тестович'),
    ROW('9999', '000000', 'Тест', '2025-01-01'),
    'test_cascade@mail.ru',
    'test_cascade',
    'test123',
    '1990-01-01'
);
-- 16. Добавляем телефоны для тестового клиента
INSERT INTO phone (client_id, phone_number)
VALUES
    ((SELECT id FROM client WHERE login = 'test_cascade'), '+79990000001'),
    ((SELECT id FROM client WHERE login = 'test_cascade'), '+79990000002');
-- 17. Проверяем, что телефоны добавлены
SELECT * FROM phone
WHERE client_id = (SELECT id FROM client WHERE login = 'test_cascade');
-- 18. Удаляем тестового клиента - телефоны удалятся автоматически (CASCADE)
DELETE FROM client
WHERE login = 'test_cascade';
-- 19. Проверяем, что телефоны удалены каскадно
SELECT * FROM phone
WHERE client_id = (SELECT id FROM client WHERE login = 'test_cascade');
```

Запросы 15 и 16 создают тестового клиента с логином 'test_cascade' и добавляют ему два номера телефонов (+79990000001 и +79990000002) для

демонстрации каскадного удаления. Запрос 17 проверяет успешное добавление телефонов, возвращая 2 записи с client_id=21. Запрос 18 удаляет тестового клиента из таблицы client. Благодаря ограничению ON DELETE CASCADE, заданному при создании внешнего ключа в таблице phone, все связанные телефоны удаляются автоматически без явного выполнения DELETE для таблицы phone. Запрос 19 подтверждает корректность каскадного удаления — результат выборки пуст (0 строк), что свидетельствует об автоматическом удалении всех телефонов вместе с клиентом.

